MAIA

Intelligent Monitoring & Auto-Healing Platform

Your Monitoring Assistant for Intelligent Operations

System Specification Document

Version 2.1 · May 2026

Table of Contents

1 Executive Summary

MAIA — **Monitoring Assistant & Intelligent Automation** — is a rule-based intelligent automation platform for data pipeline operations. It monitors any configured job type — DTSX packages, Windows services, executable processes, or any system whose errors can be scanned from log files, database tables, or API endpoints.

MAIA detects failures, classifies them by error type using configurable pattern matching rules, generates fix recommendations from a policy catalogue, and either resolves them automatically via auto-heal or routes them to an operator for review — all through a modern Angular web dashboard.

Every classification, recommendation, and fix execution is deterministic and traceable to an explicit configuration record. There are no opaque model decisions — the system is fully auditable, offline-capable, and predictable in production.

Faster Detection	Less Manual Work	Full Audit Trail	Self-Service Config
Seconds vs. hours of manual log review	Configurable auto-heal rules eliminate repetitive approvals	Every action recorded immutably in AuditLog	Full CRUD via web UI — no database access needed

2 System Architecture

MAIA watches your pipelines, understands what went wrong, knows how to fix it, and either fixes it automatically or tells the operator exactly what to do.

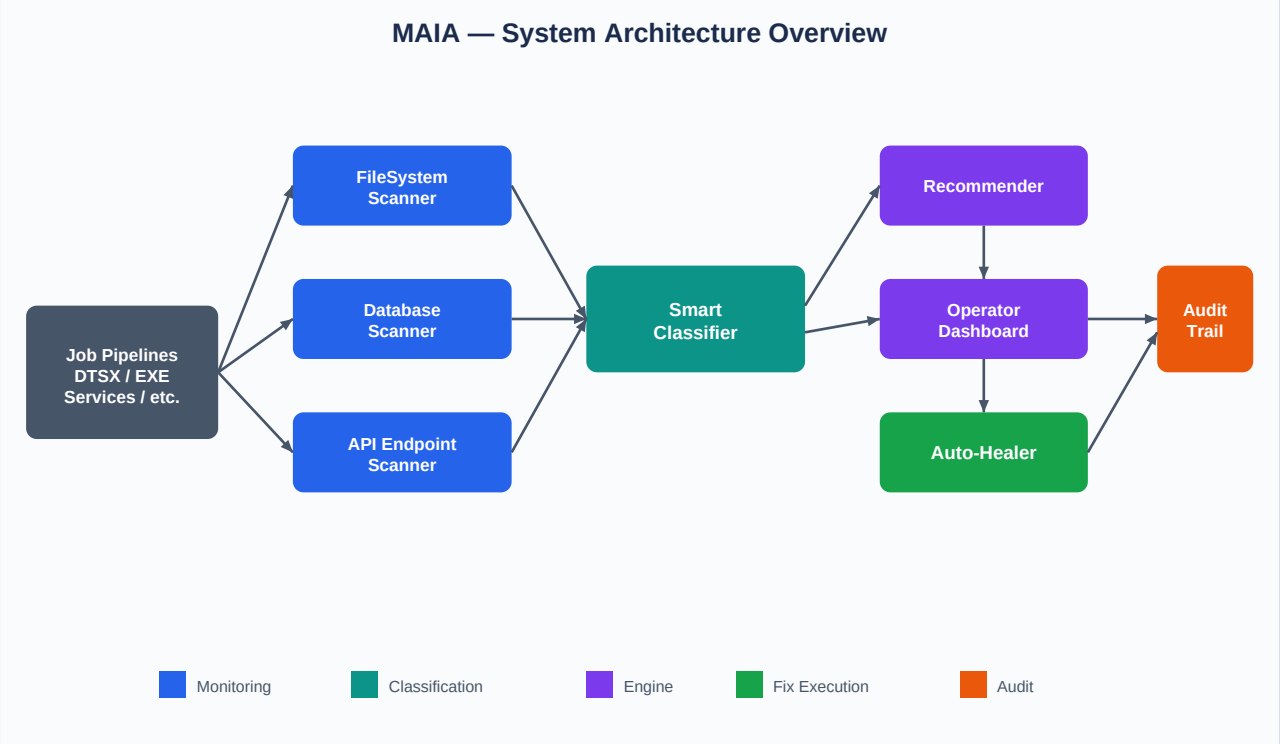

Figure 1 — MAIA System Architecture Overview

Technology Stack

Concern	Technology
Backend	C# / .NET 8 / ASP.NET Core / Entity Framework Core
Frontend	Angular 20.2, TypeScript 5.9, RxJS 7.8 — standalone components, lazy-loaded routes
Database	SQL Server (EF Core migrations)

Background Services	MonitoringWorker (lease-coordinated scan + drain), ScanHistoryRetentionWorker (retention sweep)
Deployment	Designed for containerised deployment. Multi-instance safe via MonitoredJobLeases table.
Testing	xUnit (backend), Karma / Jasmine (frontend)

3 Key Stakeholders & User Roles

Role	Who They Are	What They Do
Operator	Data / Support Engineer	Reviews failures, approves/rejects fixes, sets auto-heal policy
Admin	Senior Engineer / Team Lead	Configures monitored jobs, classification rules, fix policies, error types
System (Auto)	MAIA itself	Runs scans, classifies failures, generates recommendations, executes auto-heal fixes
Auditor	Compliance / Management	Reviews the immutable audit log for compliance

4 Intelligence Model & Future Direction

Current: deterministic rule-based automation

MAIA's classification and recommendation engine is entirely rule-based. No machine learning or large language model is used. Every decision is traceable to a configuration record:

Capability	How It Works
Classification	RuleBasedClassifier matches error messages against ClassificationRule patterns (wildcard or regex). First match by Priority assigns JobType + ErrorType.
Recommendation	GenerateSuggestionsUseCase looks up FixPolicyRule keyed by (JobTypeId, ErrorTypeId). The matching policy becomes the recommendation.
Fix Execution	ExecuteFixesUseCase dispatches to the executor matching ActionType (ApiCall, StoredProcedure, Script, Manual). Every execution writes to FixExecutionLog and AuditLog.
Auto-Heal	AutoFixAvailable is a snapshot of FixPolicyRule.IsAutoHealEligible at generation time. The live toggle mutates the policy and affects future recommendations only.

Why deterministic is the right choice now

Rule-based automation is intentional: the system must run in environments with no internet access. Every fix must be explainable and auditable against a known policy. Operators retain full control.

Path to AI-augmented monitoring (next generation)

MAIA's architecture is designed for LLM/ML integration as a natural extension:

Capability	Description
Novel error classification	LLM-assisted classification for patterns that don't match existing rules. RuleBasedClassifier runs first; unmatched failures fall through to an LLM classifier.
Dynamic fix suggestions	LLM generates candidate fix actions for failure types with no configured FixPolicyRule. Operator approval still required.
Anomaly detection	ML-based detection of subtle patterns (row count drift, latency trends) beyond threshold check rules.
Natural language config	Operators describe monitoring intent in plain language; system generates draft rules for admin review.

Extension Points Already in Place

- IClassificationStrategy — swap or chain classifiers (RuleBasedClassifier today, LLM tomorrow)
- IFixCatalogue — DbFixCatalogue is current; an LLM-backed catalogue can be added alongside it
- IScanStrategy — new scan strategies can be registered without modifying existing ones

5 Operator Workflow

The primary day-to-day workflow for an operator responding to a pipeline failure.

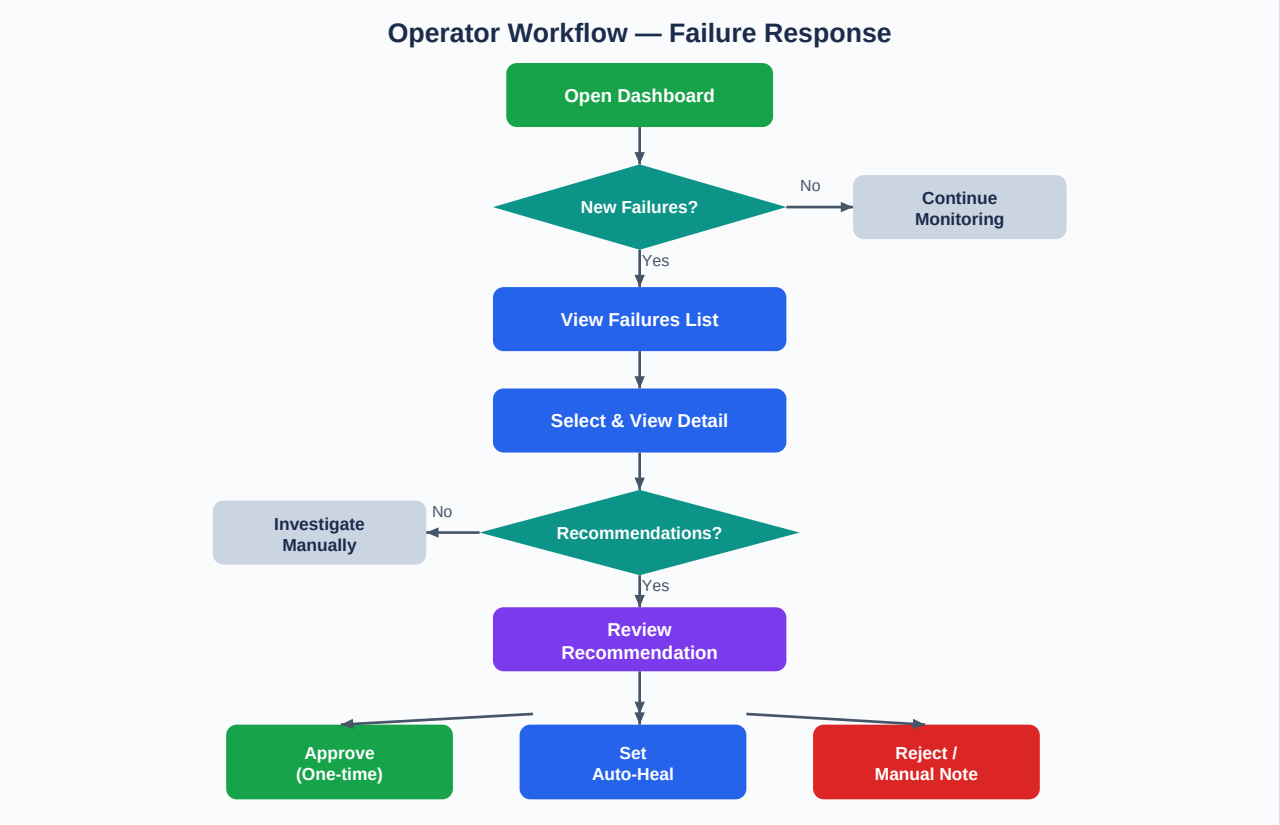

Figure 2 — Operator Failure Response Workflow

6 Auto-Heal Workflow

Once auto-heal rules are configured, MAIA resolves failures without any human intervention. The MonitoringWorker drains pending auto-heal recommendations on each tick after the parallel scan batch completes.

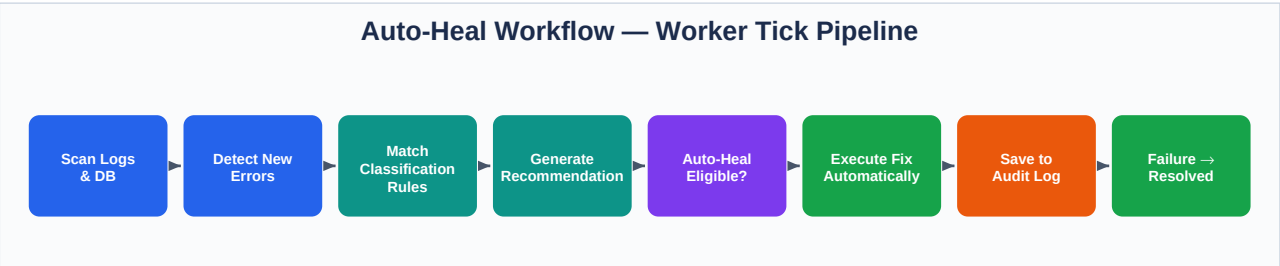

Figure 3 — Auto-Heal Background Process

Auto-Heal is Enabled When…

- A Fix Policy Rule for that (JobType, ErrorType) has IsAutoHealEligible = true, configured by an admin
- An operator has approved a previous recommendation and toggled auto-heal on the Recommendations screen

7 Configuration Management Workflow

How an admin sets up a new job for monitoring through the web UI — no database access required.

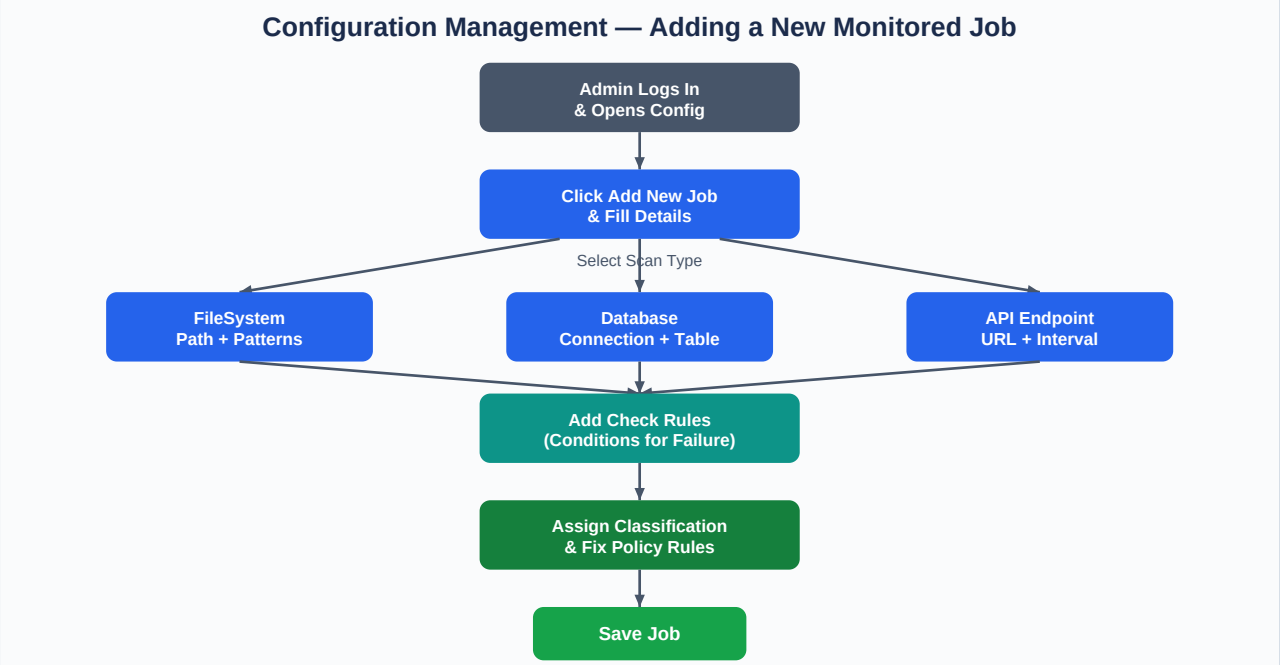

Figure 4 — New Job Configuration Workflow

8 Feature Status

All core features are built and operational as of v2.1 (May 2026).

Monitoring

FileSystem scan — ErrorKeyword check rules, wildcard stripping, byte-offset watermarks	Built
Database scan — ColumnRange + ValueEquals rules, filtered watermark advancement	Built
API Endpoint scan strategy	Built
Incremental watermarks (file offset + DB row ISO-format timestamp)	Built
Per-ScanType lease duration (FS=300s, DB=1800s, API=60s)	Built
Per-tick ScanRunHistory + 30-day retention worker + admin cleanup endpoint	Built

Classification

RuleBasedClassifier — Priority-ordered, per-job rule overrides, wildcard patterns	Built
(JobTypeId, ErrorTypeId) dual-key policy lookup everywhere	Built
ErrorType CRUD (soft-delete) via /api/config/error-types	Built

Fix Execution

ApiCallExecutor, StoredProcedureExecutor, SqlScriptExecutor, ManualActionExecutor	Built
SqlScriptExecutor: target connection from MonitoredJob + {failureId}/{sourceId} placeholders	Built
Detect-then-drain pattern — scan strategies never execute fixes	Built
Centralised drain: startup + post-tick + approve endpoint + manual trigger	Built

Operator UI

Dashboard — KPI cards, failures table, status badges	Built
Failure detail + recommendations with Approve / Reject actions	Built
Recommendations screen — confidence scores, auto-heal toggle, auto-run badge	Built
MonitoredJobs config — 3-tab panel (Scan Rules / Classification / Fix Options)	Built
Classification Rules screen — filter bar, CRUD drawer	Built
Error Types screen — CRUD (Code, DisplayName, Severity, Active)	Built

Infrastructure

Lease-coordinated MonitoringWorker — multi-instance safe, READPAST + UPDLOCK	Built
ScanHistoryRetentionWorker — scheduled + on-demand sweep	Built
GlobalExceptionHandler — RFC 7807 ProblemDetails	Built
AuditLog — immutable append-only trail	Built

Planned — Next Generation

LLM-assisted classification for novel/unmatched error patterns	Planned
Dynamic fix suggestions via language model	Planned
ML-based anomaly detection	Planned
Authentication / role-based authorization	Planned

9 Glossary

Term	Definition
MonitoredJob	A configured pipeline/process that MAIA watches. Can be any job type — DTSX, EXE, Windows Service, etc. Has scan type, connection settings, and polling interval.
ScanCheckRule	A condition evaluated during a scan (ColumnRange, ValueEquals, ErrorKeyword). Triggers a JobFailure when breached.
ClassificationRule	A pattern that assigns a JobType and ErrorType to a detected failure. Supports * wildcards and regex. Priority-ordered; first match wins.
JobFailure	A detected failure instance — one error event from a monitored job.
Recommendation	A suggested fix action generated from a FixPolicyRule lookup. Carries ConfidenceScore and AutoFixAvailable snapshot.
FixPolicyRule	Admin-configured rule keyed by (JobTypeId, ErrorTypeId): ActionType, ActionPayload, IsAutoHealEligible.
Auto-Heal	Automatic fix execution without operator approval, governed by IsAutoHealEligible. Every auto-heal is logged.
Detect-then-Drain	Scan strategies detect + classify + suggest only. Fix execution is centralised in the worker drain and approve endpoint.
Watermark	Saved position (file byte offset or DB row ISO timestamp) enabling incremental scanning.
ScanRunHistory	Append-only table: one row per completed scan. Timing, outcome, failure/classification/recommendation counts. Pruned after 30 days.
AuditLog	Immutable append-only log of all significant system actions.

TECHNICAL APPENDIX

Architecture · Workers · Data Flow · Fix Engine · Scan Strategies · Database Schema · API · NFR

A Appendix — Clean Architecture Layers

All dependencies point inward. Core has zero external dependencies.

Layer	Components	Responsibility
Presentation (AIEngineAPI)	DataController, ConfigController, ClassificationController, FixController, PipelineController, RecommendationsController, JobScanController, AdminController	HTTP endpoints, RFC 7807 errors
Application (Use Cases)	ClassifyJobsUseCase, GenerateSuggestionsUseCase, ExecuteFixesUseCase, DirectoryPipelineUseCase, ScanHistoryRetentionService	Orchestrate domain logic
Core (Domain)	Entities: JobFailure, MonitoredJob, AiRecommendation, FixPolicyRule, MonitoredJobLease, ScanRunHistory, … Interfaces: IFixEngine, IScanStrategy, IClassificationStrategy, …	Business rules & contracts
Infrastructure	SqlRepositories (12×), Scan Strategies (3×), Fix Executors (5×), Fix Handlers (4×), RuleBasedClassifier, DbFixCatalogue, 2× BackgroundServices	Data access, implements Core interfaces

B Appendix — Background Worker Pipeline

MAIA runs **two** background workers. The earlier AIClassifierWorker, LogParserWorker, and FixSuggestionWorker scaffolds were removed.

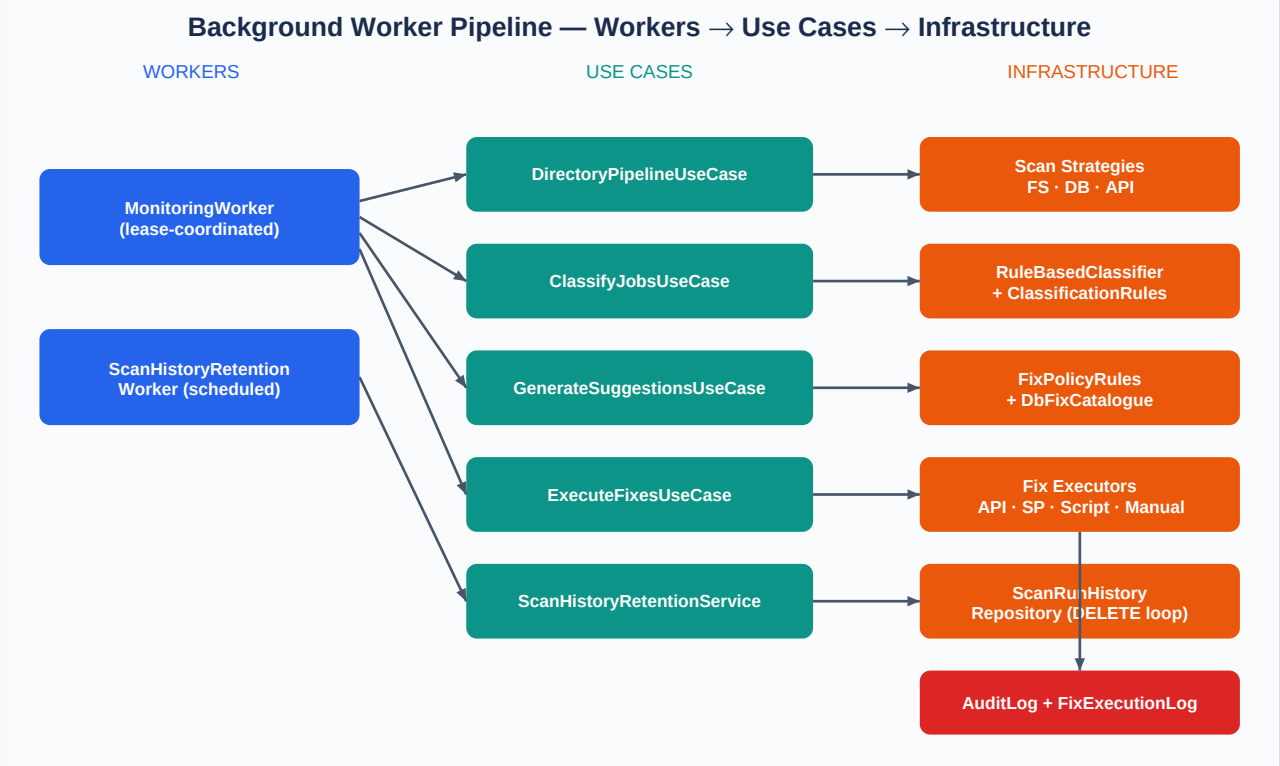

Figure B1 — Background Worker Pipeline & Use Case Orchestration

Worker	Behaviour
MonitoringWorker	Lease-coordinated BackgroundService. On each tick: (1) claims up to 4 eligible jobs atomically; (2) runs them in parallel (degree=4) with per-job DI scope and per-ScanType timeout; (3) after the batch, drains pending fixes. Writes ScanRunHistory per scan in the finally block. Also drains once at startup for restart recovery.
ScanHistoryRetention Worker	Scheduler around IScanHistoryRetentionService. Runs at startup + every CleanupIntervalHours (default 6). Deletes ScanRunHistory older than RetentionDays (default 30) in batches. Config hot-reloaded from appsettings.json.

C Appendix — End-to-End Data Flow

The complete journey of a failure event — from pipeline detection through classification, recommendation, and fix execution to resolution.

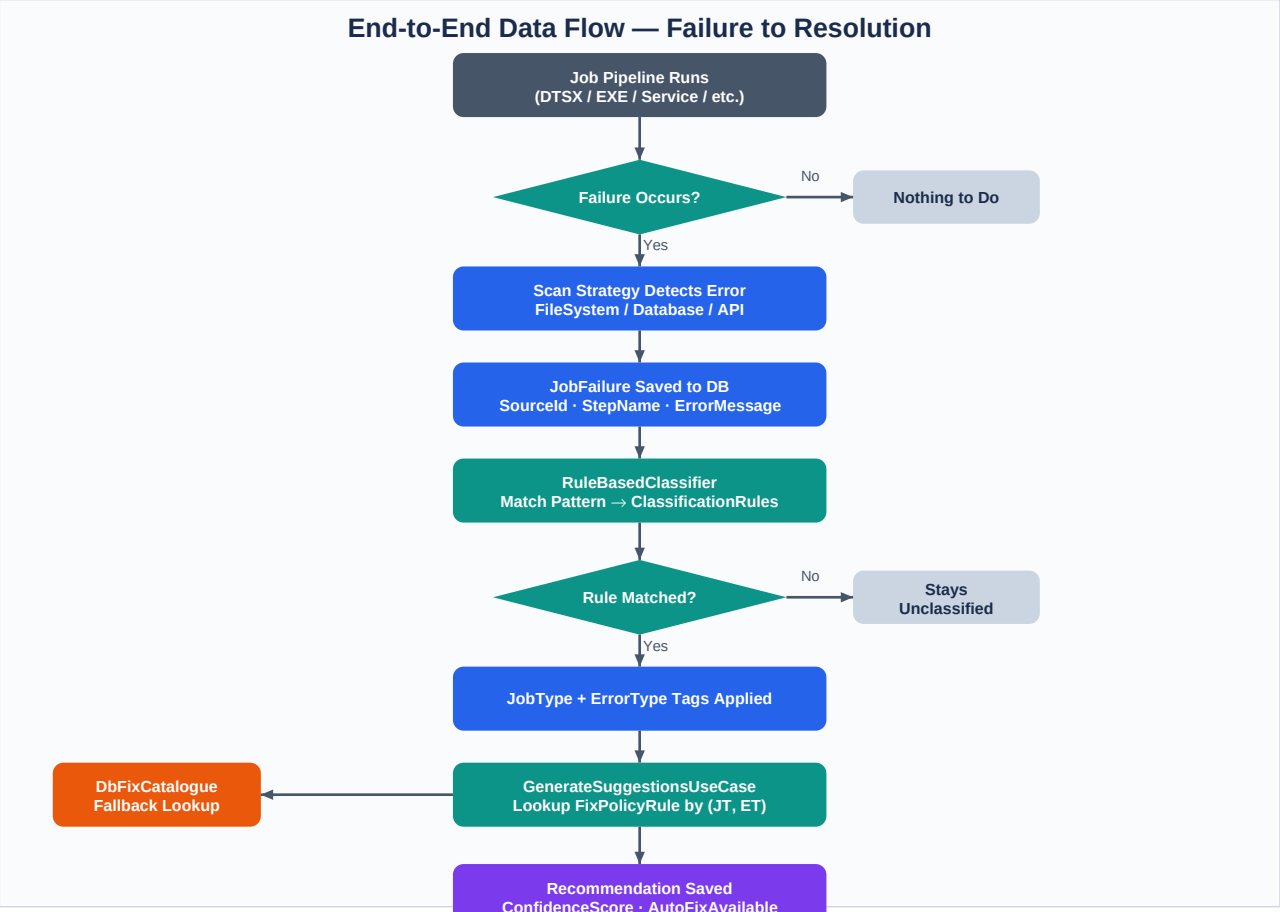

Figure C1 — End-to-End Data Flow: Pipeline Failure to Resolution

D Appendix — Fix Execution Engine

ExecuteFixesUseCase dispatches to the appropriate executor based on FixPolicyRule ActionType. If no policy exists, it falls back to FixCategory-based handlers.

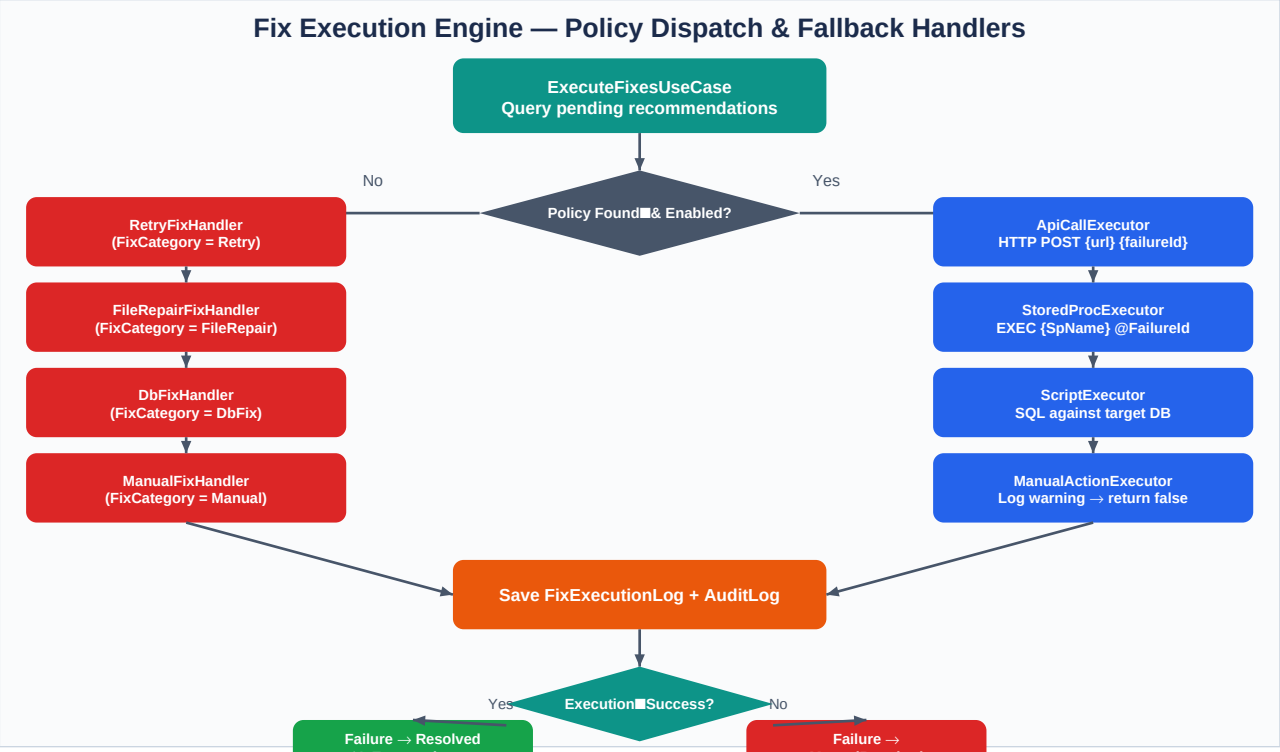

Figure D.1 — Fix Execution Engine: Policy Dispatch & Fallback Handlers

E Appendix — Scan Strategies

Each MonitoredJob is configured with one of three scan strategies. All strategies save failures to SQL Server and use watermarks to prevent re-processing.

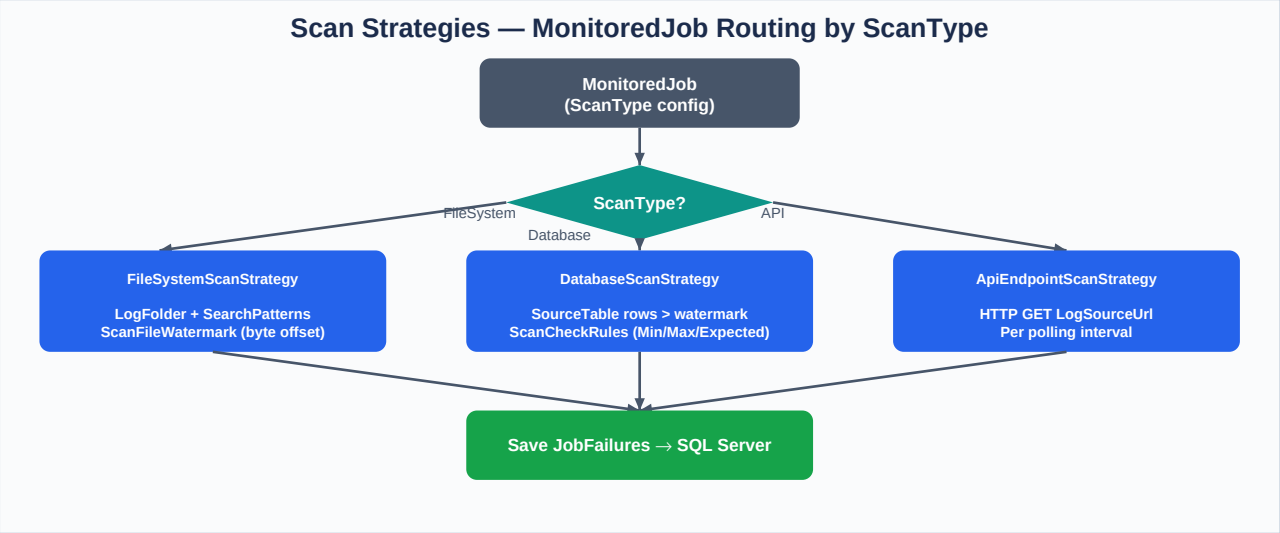

Figure E1 — Scan Strategy Selection & Watermark Tracking

Strategy	Input	Watermark	Check Rules
FileSystemScanStrategy	Log files in LogFolder (SearchPatterns e.g. *.log)	ScanFileWatermark (byte offset)	ErrorKeyword check rules
DatabaseScanStrategy	SourceTable rows WHERE col > watermark	ScanDbWatermark (ISO datetime2 string)	ColumnRange, ValueEquals
ApiEndpointScanStrategy	HTTP GET response from LogSourceUrl	Per polling interval (no persistent watermark)	Response body error signal detection

F Appendix — Database Schema

MAIA uses SQL Server with 17 tables. AuditLog is append-only and never updated or deleted.

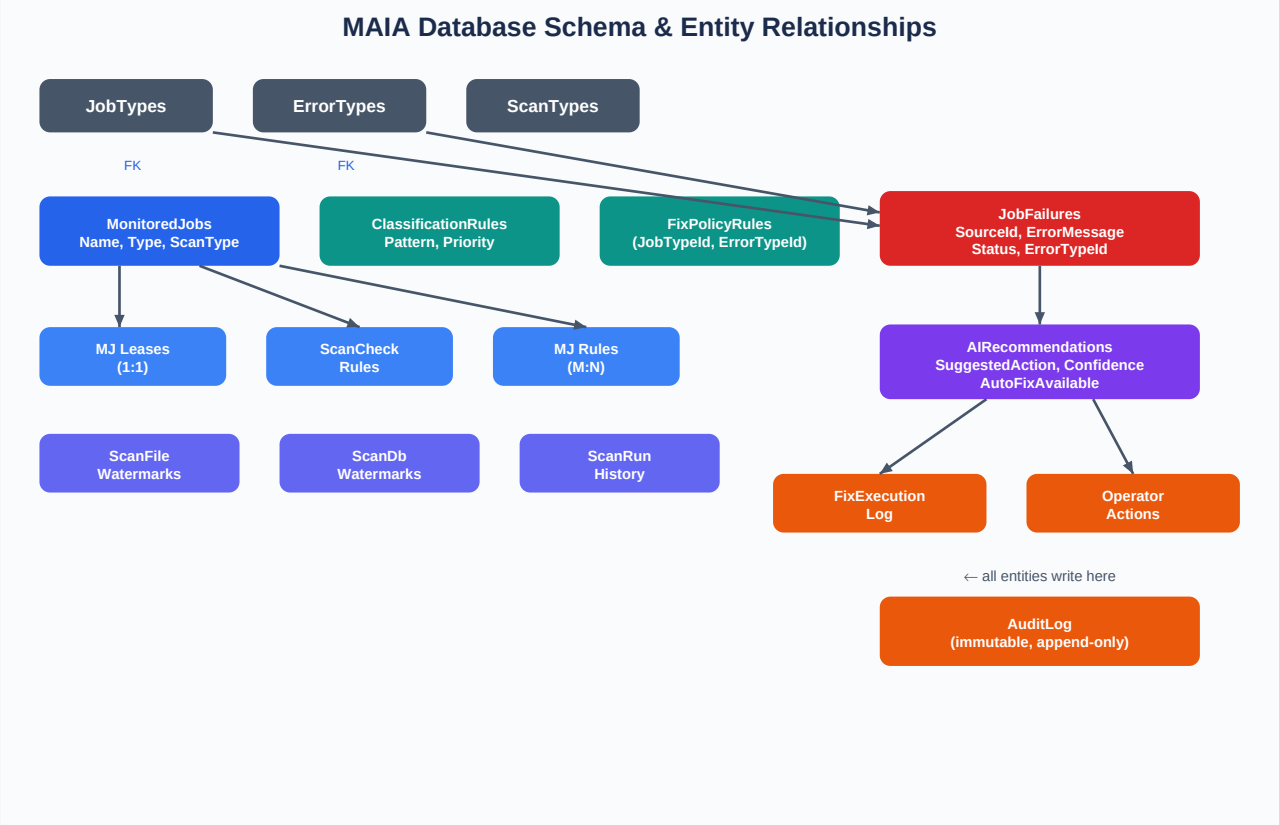

Figure F1 — MAIA Database Schema & Entity Relationships

Table	Description
MonitoredJobs	Core config: name, type, scan type, connection/path, polling interval, IsActive
MonitoredJobRules	M:N join: MonitoredJob ↔ ClassificationRule
MonitoredJobLeases	1:1 with MonitoredJob. Runtime lease state. ON DELETE CASCADE.
ScanTypes	Lookup: FileSystem, Database, ApiEndpoint + LeaseDurationSeconds
ScanCheckRules	Per-job failure conditions: ColumnRange, ValueEquals, ErrorKeyword
ScanDbWatermarks	Incremental DB scan watermark (ISO-format datetime2 string)
ScanFileWatermarks	Incremental file scan watermark (byte offset)
JobTypes	Lookup: DTSX, EXE, Service, …
ErrorTypes	Lookup: Timeout, FileNotFound, … Soft-delete. Code unique.
ClassificationRules	Match pattern → (JobTypeId, ErrorTypeId). Priority + Confidence.
JobFailures	Detected failure instance. SourceId, StepName, ErrorMessage, Status.
AIRecommendations	Fix suggestion per failure. ConfidenceScore, AutoFixAvailable, IsExecuted.
FixPolicyRules	Keyed by (JobTypeId, ErrorTypeId). ActionType, ActionPayload, IsAutoHealEligible.
FixExecutionLog	Result of each fix attempt: executor type, IsSuccess, ErrorMessage.
OperatorActions	Human approve/reject records.
AuditLog	Immutable trail: EntityType, EntityId, Action, PerformedBy, Details, Timestamp.

ScanRunHistory	Append-only per-tick scan log. INT IDENTITY PK. Pruned after 30 days.

G Appendix — API Endpoint Catalog

All HTTP endpoints exposed by AIEngineAPI.

DataController — Read-Only Queries

Method	Endpoint	Description
GET	/api/data/jobs?page=&pageSize=	Paginated failures list
GET	/api/data/failures/{id}/status	Failure detail + recommendations
GET	/api/data/recommendations?page=&pageSize=	Paginated recommendations (+ live policy snapshot)
GET	/api/data/monitored-jobs	All active monitored jobs
GET	/api/data/scan-runs?monitoredJobId=&outcome=&…	Scan run history (paged, max 200)

ConfigController — CRUD

Method	Endpoint	Description
CRUD	/api/config/monitored-jobs[/{id}]	MonitoredJob CRUD
CRUD	/api/config/scan-rules[/{id}]	ScanCheckRule CRUD
CRUD	/api/config/classification-rules[/{id}]	Global ClassificationRule CRUD
CRUD	/api/config/fix-policy-rules[/{id}]	FixPolicyRule CRUD (?jobTypeId= filter)
GET	/api/config/job-types	JobType lookup
CRUD	/api/config/error-types[/{id}]	ErrorType CRUD. DELETE is soft-delete.

RecommendationsController — Operator Actions

Method	Endpoint	Description
POST	/api/recommendations/{id}/approve	Set OperatorApproved=true + synchronous drain
POST	/api/recommendations/{id}/reject	Set OperatorApproved=false. No execution.

JobScanController — On-Demand Triggers

Method	Endpoint	Description
GET/POST	/api/jobscan/{monitoredJobId}	Trigger scan for one job
GET/POST	/api/jobscan/by-name/{name}	Trigger scan by name
POST	/api/jobscan/scan-all	Scan all active jobs
POST	/api/jobscan/classify-pending	Re-classify + drain pending fixes

Processing + Admin Controllers

Method	Endpoint	Description
POST	/api/classification/classify	Run ClassifyJobsUseCase
POST	/api/fix/execute-fixes	Manual global drain
POST	/api/pipeline/run-pipeline	Run full directory pipeline (no execute)
POST	/api/admin/scan-history/cleanup	On-demand retention sweep

H Appendix — Non-Functional Requirements

Concern	Approach
Reliability	Worker restarts on crash. Lease expiry lets another instance steal abandoned work. Startup drain recovers queued approvals.
Offline / Air-Gap	No cloud dependency at runtime. All rules stored in SQL Server. Scan strategies target local paths or configured URLs.
Horizontal Scale	Multiple worker instances run safely via MonitoredJobLeases (READPAST prevents lock contention).
Auditability	Every fix writes to FixExecutionLog + immutable AuditLog. Operator actions and auto-heal both logged.
Error Handling	GlobalExceptionHandler returns RFC 7807 ProblemDetails on all unhandled exceptions.
Extensibility	New scan type: implement IScanStrategy. New fix executor: implement IFixActionExecutor. LLM classifier: implement IClassificationStrategy.
Incremental Scanning	File watermarks (byte offset) and DB watermarks (ISO datetime2) prevent re-processing. DB watermark advances within rule-filtered rows only.
Retention	ScanRunHistory pruned after 30 days. Batch delete with configurable size and inter-batch delay. Config hot-reloaded.

I Appendix — Known Follow-ups

Tracked gaps — none block normal operation.

Item	Detail
Authentication	No authn/authz. All endpoints currently open. OperatorId is hardcoded. Needs role-based auth before production.
AuditLog gap	ConfigController PUTs/POSTs may not write AuditLog. Config changes affecting auto-execution should be auditable.
DbFixHandler stub	Returns false — failures fall to ManualRequired. Either implement or emit a clearer diagnostic message.
Drain race condition	ExecuteFixesUseCase has no per-recommendation claim. Concurrent drains can double-execute. Accepted; revisit if observed.